

Terraform AWS Guide

From Zero to Production

Terraform AWS Guide -- From Zero to Production

Complete, copy-paste-and-run examples for provisioning AWS with Terraform.

Setup

Install Terraform, authenticate via AWS CLI profile, env vars, or OIDC role.

Project 1: S3 + CloudFront Static Site

Simple static website behind CloudFront CDN. HCL below:

```
resource "aws_s3_bucket" "site" { bucket = var.bucket_name }
```

```
resource "aws_s3_bucket_public_access_block" "site" {  
  bucket = aws_s3_bucket.site.id  
  block_public_acls      = true  
  block_public_policy    = true  
  ignore_public_acls    = true  
  restrict_public_buckets = true  
}
```

```
resource "aws_s3_object" "index" {  
  bucket      = aws_s3_bucket.site.id  
  key         = "index.html"  
  source      = "${path.module}/site/index.html"  
  content_type = "text/html"  
  etag        = filemd5("${path.module}/site/index.html")  
}
```

```
resource "aws_cloudfront_origin_access_control" "site" {  
  name = "s3-oac-${var.bucket_name}"  
  origin_access_control_origin_type = "s3"  
  signing_behavior = "always"  
  signing_protocol = "sigv4"  
}
```

```
data "aws_iam_policy_document" "cloudfront_read" {  
  statement {  
    actions = ["s3:GetObject"]  
    resources = ["${aws_s3_bucket.site.arn}/*"]  
    principals {  
      type = "Service"  
      identifiers = ["cloudfront.amazonaws.com"]  
    }  
    condition {  
      test = "StringEquals"  
      variable = "aws:SourceArn"  
      values = [aws_cloudfront_distribution.site.arn]  
    }  
  }  
}
```

```

resource "aws_s3_bucket_policy" "cf" {
  bucket = aws_s3_bucket.site.id
  policy = data.aws_iam_policy_document.cloudfront_read.json
}

```

```

resource "aws_cloudfront_distribution" "site" {
  enabled          = true
  default_root_object = "index.html"
  aliases          = var.domain_name != "" ? [var.domain_name] : []
  origin {
    domain_name      = aws_s3_bucket.site.bucket_regional_domain_name
    origin_id        = "s3-${var.bucket_name}"
    origin_access_control_id = aws_cloudfront_origin_access_control.site.id
  }
  default_cache_behavior {
    allowed_methods  = ["GET", "HEAD"]
    cached_methods  = ["GET", "HEAD"]
    target_origin_id = "s3-${var.bucket_name}"
    viewer_protocol_policy = "redirect-to-https"
    forwarded_values {
      query_string = false
      cookies { forward = "none" }
    }
  }
  min_ttl = 0; default_ttl = 3600; max_ttl = 86400
}
restrictions { geo_restriction { restriction_type = "none" } }
viewer_certificate {
  cloudfront_default_certificate = var.domain_name != "" ? false : true
}
}

```

****Deploy**:** mkdir site; terraform init; terraform plan; terraform apply

Project 2: VPC from Scratch

No default VPC. Build every component manually.

```

resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support   = true
}

```

```

resource "aws_subnet" "public" {
  count          = length(var.azs)
  vpc_id        = aws_vpc.main.id
  cidr_block     = cidrsubnet(aws_vpc.main.cidr_block, 8, count.index)
  availability_zone = var.azs[count.index]
  map_public_ip_on_launch = true
}

```

```

resource "aws_subnet" "private" {
  count          = length(var.azs)
  vpc_id        = aws_vpc.main.id
  cidr_block     = cidrsubnet(aws_vpc.main.cidr_block, 8, count.index + 10)
  availability_zone = var.azs[count.index]
}

```

```
resource "aws_internet_gateway" "main" { vpc_id = aws_vpc.main.id }
```

```
resource "aws_eip" "nat" { domain = "vpc" }
```

```
resource "aws_nat_gateway" "main" {  
  allocation_id = aws_eip.nat.id  
  subnet_id     = aws_subnet.public[0].id  
}
```

```
resource "aws_route_table" "public" {  
  vpc_id = aws_vpc.main.id  
  route { cidr_block = "0.0.0.0/0"; gateway_id = aws_internet_gateway.main.id }  
}
```

```
resource "aws_route_table" "private" {  
  count = length(aws_subnet.private)  
  vpc_id = aws_vpc.main.id  
  route { cidr_block = "0.0.0.0/0"; nat_gateway_id = aws_nat_gateway.main.id }  
}
```

```
resource "aws_route_table_association" "public" {  
  count          = length(aws_subnet.public)  
  subnet_id      = aws_subnet.public[count.index].id  
  route_table_id = aws_route_table.public.id  
}
```

```
resource "aws_route_table_association" "private" {  
  count          = length(aws_subnet.private)  
  subnet_id      = aws_subnet.private[count.index].id  
  route_table_id = aws_route_table.private[count.index].id  
}
```

****Module it****: Extract to modules/vpc with variables (name, cidr, azs) and outputs (vpc_id, subnet_ids).

****Key****: cidrsubnet("10.0.0.0/16", 8, n) = /24 subnets. NAT Gateway costs ~\$35/mo.

Project 3: EC2 Auto Scaling + ALB

Instances in private subnets behind an ALB in public subnets. Scale on CPU.

****Launch Template****:

```
data "aws_ami" "amazon_linux_2" { most_recent = true; owners = ["amazon"]  
filter { name = "name"; values = ["amzn2-ami-hvm-*-x86_64-gp2"] }  
}
```

```
resource "aws_launch_template" "web" {  
  name_prefix = "${var.name}-web-"  
  image_id    = data.aws_ami.amazon_linux_2.id  
  instance_type = "t3.micro"  
  vpc_security_group_ids = [aws_security_group.web.id]  
  user_data = base64encode(templatefile("${path.module}/userdata.sh", {  
    environment = var.environment  
  })))  
}
```

```

resource "aws_security_group" "web" {
name = "${var.name}-web-sg"; vpc_id = var.vpc_id
ingress { from_port = 80; to_port = 80; protocol = "tcp"
security_groups = [aws_security_group.alb.id] }
egress { from_port = 0; to_port = 0; protocol = "-1"
cidr_blocks = ["0.0.0.0/0"] }
}

```

****ALB**:**

```

resource "aws_lb" "main" {
name = "${var.name}-alb"; internal = false
load_balancer_type = "application"
security_groups = [aws_security_group.alb.id]
subnets = var.public_subnet_ids
}

```

```

resource "aws_lb_target_group" "web" {
name = "${var.name}-web-tg"; port = 80; protocol = "HTTP"; vpc_id = var.vpc_id
health_check { path = "/"; matcher = "200-299" }
}

```

```

resource "aws_lb_listener" "http" {
load_balancer_arn = aws_lb.main.arn; port = 80; protocol = "HTTP"
default_action { type = "forward"; target_group_arn = aws_lb_target_group.web.arn }
}

```

****Auto Scaling Group**:**

```

resource "aws_autoscaling_group" "web" {
name = "${var.name}-web-asg"
vpc_zone_identifier = var.private_subnet_ids
target_group_arns = [aws_lb_target_group.web.arn]
health_check_type = "ELB"
min_size = 2; max_size = 6; desired_capacity = 2
launch_template { id = aws_launch_template.web.id; version = "$Latest" }
instance_refresh { strategy = "Rolling"
preferences { min_healthy_percentage = 50 } }
}

```

```

resource "aws_autoscaling_policy" "scale_up" {
name = "${var.name}-scale-up"
autoscaling_group_name = aws_autoscaling_group.web.name
adjustment_type = "ChangeInCapacity"; scaling_adjustment = 1
}

```

```

resource "aws_cloudwatch_metric_alarm" "cpu_high" {
alarm_name = "${var.name}-cpu-high"
comparison_operator = "GreaterThanThreshold"
evaluation_periods = 2; metric_name = "CPUUtilization"
namespace = "AWS/EC2"; period = 120; statistic = "Average"
threshold = 70; alarm_actions = [aws_autoscaling_policy.scale_up.arn]
dimensions = { AutoScalingGroupName = aws_autoscaling_group.web.name }
}

```

****userdata.sh**:** yum update; install nginx; start on boot.

Project 4: RDS PostgreSQL

Stateful DB. Never in an ASG.

```
resource "aws_db_subnet_group" "main" {
  name = "${var.name}-db-subnet"; subnet_ids = var.private_subnet_ids
}
```

```
resource "aws_db_instance" "main" {
  identifier = "${var.name}-postgres"
  engine = "postgres"; engine_version = "16.3"
  instance_class = "db.t4g.micro"
  allocated_storage = 20; max_allocated_storage = 100
  storage_encrypted = true
  db_name = var.db_name; username = var.db_username
  password = random_password.db.result; port = 5432
  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.rds.id]
  publicly_accessible = false
  skip_final_snapshot = var.environment != "production"
  deletion_protection = var.environment == "production"
  backup_retention_period = var.environment == "production" ? 30 : 7
  apply_immediately = false
}
```

```
resource "random_password" "db" { length = 32; special = false }
```

```
resource "aws_secretsmanager_secret" "db" { name = "${var.name}-db-password" }
```

```
resource "aws_secretsmanager_secret_version" "db" {
  secret_id = aws_secretsmanager_secret.db.id
  secret_string = jsonencode({
    username = var.db_username; password = random_password.db.result
    host = aws_db_instance.main.address; port = 5432; dbname = var.db_name
  })
}
```

****Rules**:** no hard-coded passwords. Secrets in Secrets Manager. skip_final_snapshot=false and deletion_protection=true in production.

Project 5: IAM Roles

Least privilege principle. Every service gets a role.

```
data "aws_iam_policy_document" "ec2_assume" {
  statement { actions = ["sts:AssumeRole"]
  principals { type = "Service"; identifiers = ["ec2.amazonaws.com"] } }
}
```

```
resource "aws_iam_role" "ec2" {
  name = "${var.name}-ec2-role"
  assume_role_policy = data.aws_iam_policy_document.ec2_assume.json
}
```

```
resource "aws_iam_instance_profile" "ec2" {
name = "${var.name}-profile"; role = aws_iam_role.ec2.name
}
```

****Cross-account****: Add ExternalId condition to sts:AssumeRole trust policy.

Project 6: Lambda + API Gateway + DynamoDB

Serverless stack. PAY_PER_REQUEST DynamoDB. Python Lambda behind HTTP API Gateway.

```
resource "aws_dynamodb_table" "items" {
name = "${var.name}-items"; billing_mode = "PAY_PER_REQUEST"
hash_key = "id"; attribute { name = "id"; type = "S" }
}
```

```
resource "aws_lambda_function" "api" {
filename = data.archive_file.lambda_zip.output_path
source_code_hash = data.archive_file.lambda_zip.output_base64sha256
function_name = "${var.name}-api"; role = aws_iam_role.lambda.arn
handler = "index.handler"; runtime = "python3.12"
environment { variables = { TABLE_NAME = aws_dynamodb_table.items.name } }
}
```

```
resource "aws_apigatewayv2_api" "main" { name = "${var.name}-api"; protocol_type = "HTTP" }
```

```
resource "aws_apigatewayv2_stage" "main" {
api_id = aws_apigatewayv2_api.main.id; name = "$default"; auto_deploy = true
}
```

```
resource "aws_apigatewayv2_integration" "lambda" {
api_id = aws_apigatewayv2_api.main.id
integration_type = "AWS_PROXY"
integration_uri = aws_lambda_function.api.invoke_arn
}
```

```
resource "aws_apigatewayv2_route" "items" {
api_id = aws_apigatewayv2_api.main.id
route_key = "GET /items"
target = "integrations/${aws_apigatewayv2_integration.lambda.id}"
}
```

```
resource "aws_lambda_permission" "apigw" {
statement_id = "AllowAPIGatewayInvoke"
action = "lambda:InvokeFunction"
function_name = aws_lambda_function.api.function_name
principal = "apigateway.amazonaws.com"
source_arn = "${aws_apigatewayv2_api.main.execution_arn}/*/*"
}
```

Project 7: EKS Cluster

Production-grade K8s with managed node groups.

```
resource "aws_eks_cluster" "main" {
  name = var.name; role_arn = aws_iam_role.eks_cluster.arn; version = "1.30"
  vpc_config { subnet_ids = var.private_subnet_ids
  endpoint_private_access = true; endpoint_public_access = true }
}
```

```
resource "aws_eks_node_group" "main" {
  cluster_name = aws_eks_cluster.main.name
  node_group_name = "${var.name}-nodes"
  node_role_arn = aws_iam_role.node_group.arn
  subnet_ids = var.private_subnet_ids
  instance_types = ["t3.medium"]; capacity_type = "ON_DEMAND"
  scaling_config { desired_size = 2; max_size = 5; min_size = 1 }
}
```

****Required policies**:** AmazonEKSClusterPolicy, AmazonEKSWorkerNodePolicy, AmazonEKS_CNI_Policy, AmazonEC2ContainerRegistryReadOnly.

****Access**:** aws eks update-kubeconfig --region us-east-1 --name my-cluster

Remote State: S3 Backend + DynamoDB Locking

Shared state with locking for team environments.

```
terraform { backend "s3" { bucket = "myco-state"; key = "prod/vpc/terraform.tfstate"
region = "us-east-1"; encrypt = true; dynamodb_table = "terraform-locks" } }
```

Bootstrap once per account: S3 bucket with versioning, SSE-S3, public access blocked. DynamoDB table with LockID hash key, PAY_PER_REQUEST.

Patterns & Best Practices

****locals**:** Define common_tags and prefix once, merge everywhere via merge().

****data sources**:** aws_availability_zones, aws_route53_zone, aws_ami for dynamic lookups.

****lifecycle**:** prevent_destroy (safety), ignore_changes (password), create_before_destroy (zero-downtime).

****count vs for_each**:** Use for_each by default -- preserves resources on add/remove. Count re-creates everything on list changes.

****dynamic blocks**:** Use for repeating blocks like security group ingress rules.

****conditional resources**:** count = var.enabled ? 1 : 0 for optional resources.

****depends_on**:** Only when Terraform can't infer the dependency (e.g., S3 policy depends on CloudFront).

Project 8: Terraform CI/CD

GitLab CI pipeline:


```
stages: [validate, plan, apply]
validate: extends .terraform; script: fmt -check && validate
plan: extends .terraform; script: plan -out=plan; artifacts: [plan]
apply: extends .terraform; when: manual; script: apply plan
```

GitHub Actions: hashicorp/setup-terraform action, OIDC for AWS auth (no secrets needed).

****State isolation****: Separate state per environment via backend key (prod/, staging/).

Learning Path

1. ****S3 + CloudFront**** -- understand providers, resources, data sources, outputs
 2. ****VPC from scratch**** -- understand networking, modules, count
 3. ****EC2 + ALB + ASG**** -- understand compute, load balancing, scaling
 4. ****RDS + Secrets Manager**** -- understand stateful resources, security
 5. ****IAM**** -- understand least privilege, roles, policies
 6. ****Lambda + API Gateway**** -- understand serverless patterns
 7. ****EKS**** -- understand managed K8s on AWS
 8. ****CI/CD**** -- automate everything
-

Resources

- Terraform AWS Provider Docs: registry.terraform.io/providers/hashicorp/aws
- AWS Provider Guides: learn.hashicorp.com/tutorials/terraform
- AWS Well-Architected Framework